

Tipos de índices en PostgreSQL

Análisis técnico de B-Tree, Hash y GIN para una plataforma de e-commerce

Estudiante: Francisco Mohamad Fecha: 22 de septiembre de 2026

Resumen de decisiones

Consulta	Operación principal	Índice propuesto	Motivo principal
A: tracking_id	Igualdad exacta (=)	Hash	Está especializado en coincidencias exactas.
B: fecha_venta	Rango (BETWEEN)	B-Tree	Mantiene claves ordenadas y permite recorrer rangos.
C: propiedades	Contención en JSONB (@>)	GIN	Indexa claves y valores internos del documento.

Consulta A: búsqueda por código de seguimiento

ELECCIÓN: índice Hash sobre tracking_id.

La consulta utiliza exclusivamente igualdad exacta sobre un string alfanumérico largo. Un índice Hash transforma el valor en un código hash y permite localizar las entradas correspondientes sin conservar un orden entre las claves. Esto coincide con el patrón `WHERE tracking_id = 'ABC-123'`.

Hash no sería apropiado si en el futuro se necesitaran rangos, búsquedas por prefijo u ordenamiento. Un B-Tree también resolvería la igualdad y suele ser la alternativa general, pero, bajo el requisito indicado de búsquedas únicamente exactas, Hash representa la especialización solicitada.

```
CREATE INDEX idx_pedidos_tracking_id
ON pedidos USING HASH (tracking_id);
```

Consulta B: ventas entre fechas

ELECCIÓN: índice B-Tree sobre fecha_venta.

B-Tree conserva sus claves ordenadas y soporta operadores de comparación como `>`, `<`, `>=`, `<=` y **BETWEEN**. PostgreSQL puede ubicar el comienzo del período solicitado y recorrer únicamente las entradas hasta la fecha final, evitando inspeccionar los diez millones de registros mediante un escaneo secuencial.

```
CREATE INDEX idx_ventas_fecha_venta
ON ventas USING BTREE (fecha_venta);
```

Este índice acelera el filtro temporal. Como la consulta ordena por **monto** y no por **fecha_venta**, PostgreSQL todavía puede necesitar un paso **Sort** sobre las filas del rango obtenido. Agregar **monto** como segunda columna no garantiza el orden global por monto cuando la primera columna se consulta como rango; la conveniencia de otro índice debe comprobarse con **EXPLAIN (ANALYZE, BUFFERS)** y consultas reales.

Consulta C: filtros sobre propiedades JSONB

ELECCIÓN: índice GIN sobre **propiedades**.

Un índice GIN es invertido: descompone cada documento JSONB en elementos indexables y relaciona cada clave o valor con las filas donde aparece. De esta forma puede responder eficientemente a operadores de contención como **@>**, sin recorrer y analizar cada documento completo.

```
CREATE INDEX idx_productos_propiedades
ON productos USING GIN (propiedades);

SELECT *
FROM productos
WHERE propiedades @> '{"color": "rojo", "talla": "L"}';
```

La clase de operadores GIN predeterminada para JSONB admite búsquedas de contención y existencia. Si el sistema utilizara únicamente **@>**, **jsonb_path_ops** podría producir un índice más pequeño, pero limitaría los operadores disponibles; por eso el índice general es la propuesta más flexible para este escenario.

Reflexión sobre efectos secundarios

Impacto de crear 15 índices en inventario

Cada **INSERT** debe escribir la nueva fila y actualizar las quince estructuras de índice. Un **UPDATE** que cambie columnas indexadas también debe retirar o invalidar entradas anteriores y agregar las nuevas; los **DELETE** generan trabajo equivalente de mantenimiento. Por lo tanto, la carga de productos perdería velocidad y consumiría más CPU, operaciones de entrada/salida y WAL.

- Aumentaría el espacio en disco y la presión sobre la caché.
- Las transacciones de escritura durarían más y podrían mantener bloqueos durante más tiempo.
- **VACUUM**, copias de seguridad y tareas de mantenimiento tendrían más estructuras que procesar.
- Índices redundantes o poco usados pagarían ese costo sin mejorar consultas importantes.

La decisión correcta es indexar columnas que aparezcan con frecuencia en filtros, uniones u ordenamientos relevantes, y revisar su uso con estadísticas como **pg_stat_user_indexes**.

Uso de UPPER(nombre) con un índice simple

Un B-Tree creado sobre **nombre** almacena el valor original, no el resultado de **UPPER(nombre)**. Al aplicar una función a cada fila, la expresión de la consulta ya no coincide con la expresión indexada y, en general, PostgreSQL realizará un escaneo secuencial. Además, comparar el valor transformado permite una búsqueda sin distinción entre mayúsculas y minúsculas, mientras que el índice simple conserva la representación original.

La solución directa es crear un índice funcional que reproduzca exactamente la expresión consultada:

```
CREATE INDEX idx_clientes_nombre_upper
ON clientes (UPPER(nombre));

SELECT *
FROM clientes
WHERE UPPER(nombre) = 'JUAN';
```

Conclusión. La selección debe responder al operador y al tipo de dato: Hash para igualdad exacta, B-Tree para rangos y datos ordenables, y GIN para búsquedas internas en JSONB. El beneficio de lectura siempre debe evaluarse junto con el costo de mantener cada índice durante las escrituras.